

Sistemas Expertos

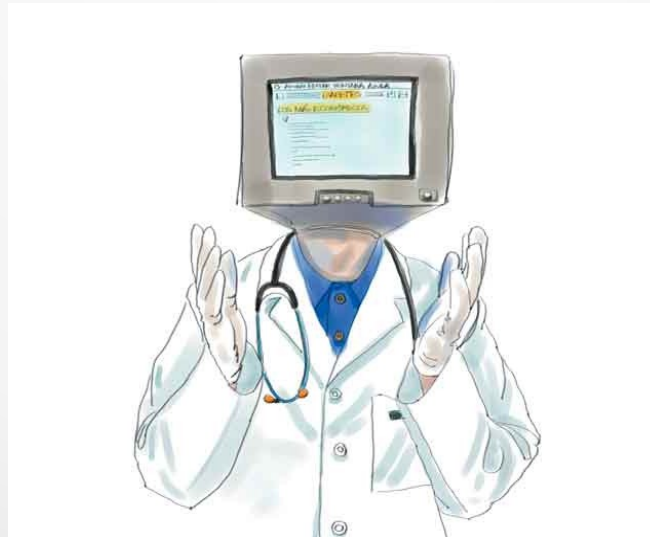


Historia

- Los sistemas expertos forman parte de la ciencia de la computación y dentro de ésta se ubican en la rama de la inteligencia artificial.
- Un doctor de la Universidad Stanford, Edward A. Feigenbaum. Formalizó que el poder de resolución de un problema en un programa de computadora viene del conocimiento de un dominio específico, no solo de las técnicas de programación y el formalismo que contiene.
- Los primeros sistemas expertos que se desarrollaron en los años 1960 eran capaces de resolver solo problemas basados en situaciones determinadas, mediante sistemas de reglas. Es a partir de los años 1970 cuando se empiezan a resolver problemas basados en situaciones inciertas.

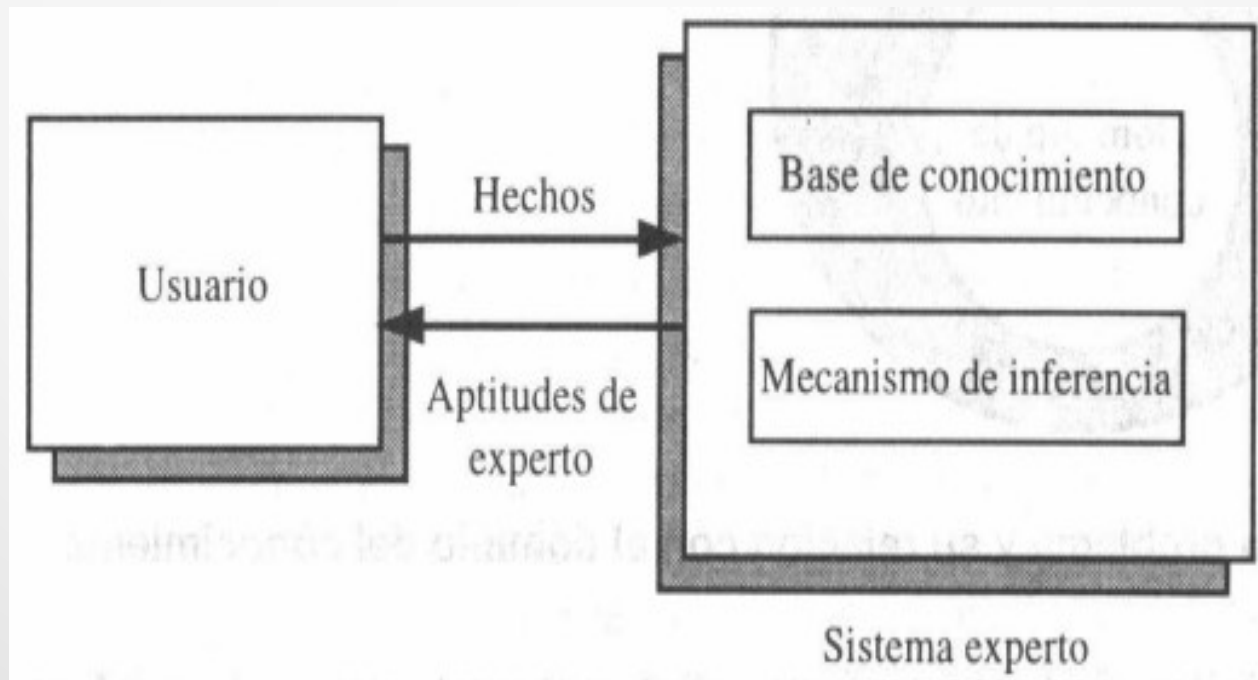
Definición

- Los sistemas expertos son programas que emulan la capacidad de tomar decisiones para resolver problemas de un especialista humano.



Sistemas Expertos

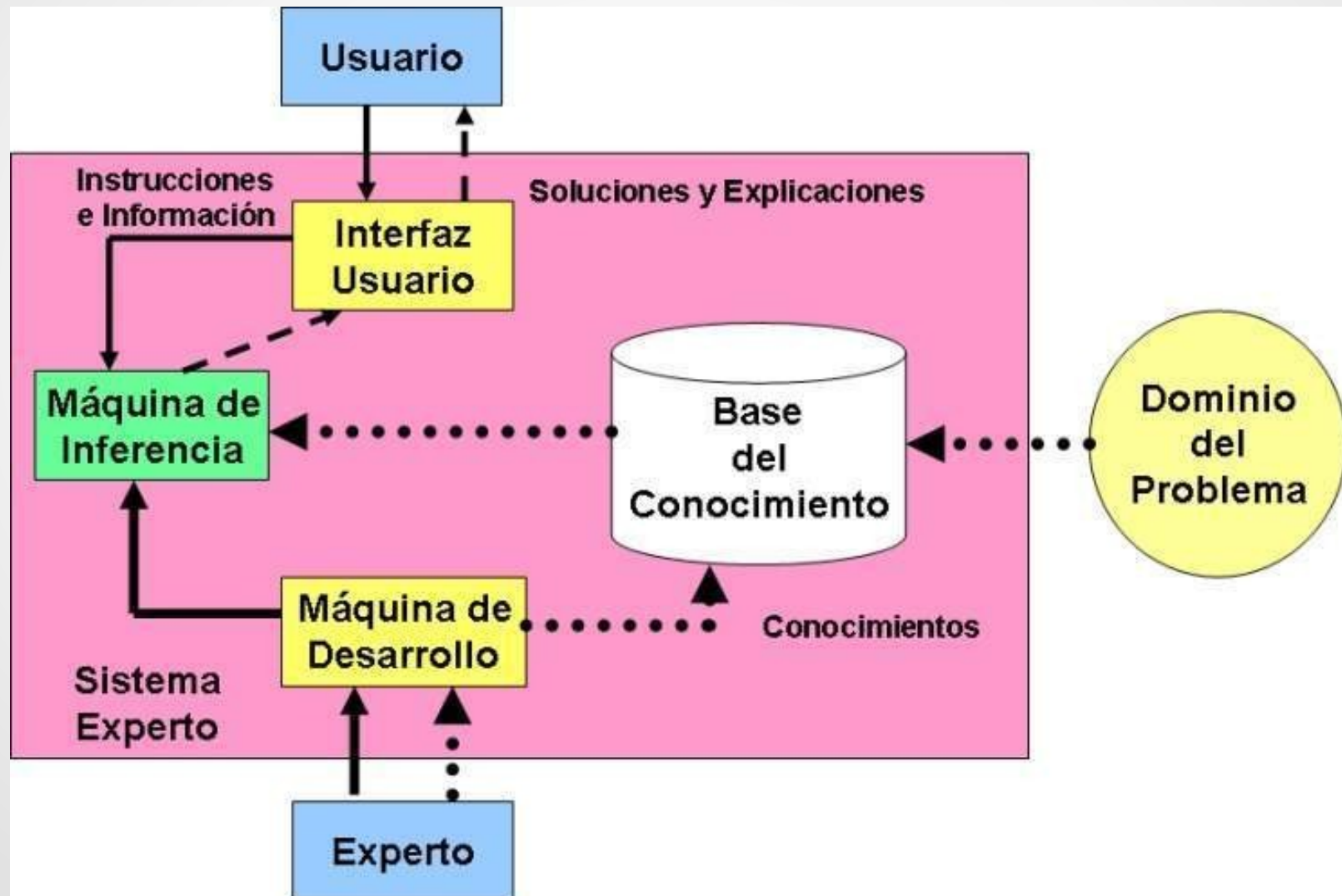
- Básicamente en un sistema experto, el usuario aporta los hechos o información y recibe consejos o experiencia como respuesta. En su interior, el sistema incluye la base de conocimiento y un mecanismo de inferencia.



Características

- Para que un sistema actúe como un verdadero experto, es deseable que reúna, en lo posible, lo más importante de las características de un experto humano. **Habilidad para adquirir conocimiento.**
- **Capacidad para resolver problemas.**
- **Fiabilidad, para poder confiar en sus resultados o apreciaciones.**
- Dada la complejidad de los problemas que usualmente tiene que resolver un sistema experto, puede existir cierta duda en el usuario sobre la validez de la respuesta obtenida. Por este motivo, es una condición indispensable que un sistema experto sea capaz de **explicar su proceso de razonamiento** o dar razón del por qué del resultado.

Componentes



Componentes

- **Base de conocimientos:** Contiene el conocimiento de los hechos y las experiencias de los expertos en un dominio determinado.
- **Mecanismo de inferencia:** Puede simular la estrategia de solución de un experto.
- **Componente explicativo:** Explica al usuario la estrategia de solución encontrada y el por qué de las decisiones tomadas
- **Interfaz de usuario:** Sirve para que este pueda realizar una consulta en un lenguaje lo más natural posible.
- **Componente de adquisición:** Ofrece ayuda a la estructuración e implementación del conocimiento en la base de conocimientos.

Ventajas

- Mayor disponibilidad: La experiencia esta disponible en todo momento.
- Peligro reducido: Los S.E. pueden usarse en ambientes peligrosos para un ser humano.
- Permanencia: La experiencia dura indefinidamente, a diferencia de un especialista que puede renunciar por ejemplo.
- Mayor confiabilidad: Al proporcionar una segunda opinión para la toma de decisiones.
- Respuesta rápida.

Limitaciones

- Reprogramación en actualizaciones.
- Elevado costo en dinero y tiempo.
- No resuelven problemas generales.
- Lenguaje natural: Con un experto humano podemos mantener una conversación informal mientras que con un SE no.
- Falta de flexibilidad para incorporar conocimiento.

Sistemas Expertos

Sistema MYCIN para diagnóstico de meningitis y bacteremia (infecciones bacterianas)

SI

El medio de cultivo es la sangre, y

La identidad del organismo no es conocida con certeza, y

La coloración del organismo es gramnegativa, y

La morfología del organismo es de bastoncillo, y

El paciente presenta elevación de la temperatura

ENTONCES

Hay una débil evidencia que sugiere(.4) que la identidad del organismo es pseudomona.

JESS

- JESS (Java Expert System Shell) es un shell de programación para la construcción de S.E. basado en CLIPS para la plataforma Java. Su funcionamiento se basa en reglas y usa el algoritmo Rete. Licencia comercial (educación: requiere registro).
- CLIPS (C Language Integrated Production System) es otra herramienta para el desarrollo de S.E. que ofrece un entorno completo para su construcción basado en reglas y objetos. CLIPS fue creado por la NASA.

JESS

**Aplicaciones Java +
Razonamiento de un Sistema Experto (reglas)**

JESS

Componentes:

- Memoria de trabajo: hechos (facts). Cada hecho tiene una plantilla (template). Cada plantilla tiene un nombre y un conjunto de slots.
- Base de reglas. Las reglas sólo pueden reaccionar ante agregado, eliminación o cambios en la memoria de trabajo.
- Motor de inferencias RETE (algoritmo de reconocimiento de patrones).

JESS

- Para utilizarlo se deben especificar 'reglas'.
- Es necesario proveer 'hechos' de acuerdo al problema.
- Cuando se ejecuta el motor, las reglas pueden ser disparadas.
- Las reglas pueden crear nuevos hechos, o pueden realizar cualquier tarea que se pueda realizar con Java.

JESS

- Elementos básicos: letras, dígitos, booleanos, números y strings.
- Listas.
- Todo el código en Jess tiene la forma de función.
- Ejemplo de llamada a función: `(+ (+ 2 3) (* 3 3))`
- Definición de función:

```
(deffunction max (?a ?b)
  ( if (> ?a ?b) then ?a
    else ?b )
)
```

JESS

- Variables: Identificadores que comienzan con '?'.
(bind ?a 123)
- Con el comando **facts** es posible ver una lista de todos los hechos de la memoria de trabajo.
- Los hechos se pueden agregar mediante las funciones **assert**, **add** y **definstance**.

JESS: Hechos.

- Se deben definir mediante deftemplate:

```
(deftemplate auto "un auto específico"  
  (slot marca) (slot modelo) (slot color (default blanco)) )
```

- Se agregan mediante deffacts o assert:

```
(defacts (auto (marca Ford) (modelo Explorer)) )  
(assert (auto (marca Ford) (modelo Explorer)) )
```

JESS: Reglas.

- Una regla es similar a una sentencia if...then...
- Las reglas se ejecutan cuando se satisface la parte del if.
- Ejemplo:

```
(deftemplate persona (slot nombre) (slot edad) )
```

```
(defrule personaJose
```

```
  (persona (nombre "jose") (edad 28) )
```

```
  => (printout t "encontre a jose" crlf)
```

```
)
```

JESS: Reglas.

- (defrule edadJose
 (persona (nombre "Jose") (edad ?e))
 =>
 (printout t "Jose tiene " ?e " años" crlf)
)

**(defrule <nombre-regla>
 (patrón 1) ... (patrón N)
 =>
 (acción 1) ... (acción N)
)**

JESS vs PROLOG

- JESS y Prolog son lenguajes declarativos, es decir, basados en las matemáticas y en la lógica
- La diferencia más importante y que tiene bastante que ver con las anteriores es que Prolog se basa en el encadenamiento hacia atrás, mientras que JESS usa el encadenamiento hacia adelante.
- En Prolog, si se quiere saber de nuevo un resultado, se debe volver a calcular. Mientras, en JESS los resultados (hechos que se obtienen) quedan almacenados.
- El encadenamiento hacia atrás (backwards chaining) parte de una hipótesis inicial y luego intenta demostrarla con la información que se tiene.
- El encadenamiento hacia adelante (forwards chaining) consiste en aplicar al conocimiento base otro conocimiento (las reglas) para obtener nuevos resultados.